

No strongly regular graph $\text{srg}(99, 14, 1, 2)$ admits an automorphism
of order seven

Blueprint of the formal proof

Utku Okur

July 23, 2026

Contents

1	Strongly regular graphs and the statement	2
2	From an order-seven automorphism to a canonical one	3
3	The orbit quotient and the propositional encoding	4
4	Completeness of the normalisation	6
5	The partition of the valuation space	7
6	Certificates	8

Chapter 1

Strongly regular graphs and the statement

Definition 1 (Strongly regular graph). A finite graph Γ on n vertices is *strongly regular with parameters* (n, k, λ, μ) if it is k -regular, any two adjacent vertices have exactly λ common neighbours, and any two distinct non-adjacent vertices have exactly μ common neighbours.

Lean: `SimpleGraph.IsSRGWith` ✓

Theorem 2 (Main theorem). *Let Γ be a strongly regular graph with parameters $(99, 14, 1, 2)$, on any vertex set. Then $7 \nmid |\text{Aut } \Gamma|$; in particular Γ has no automorphism of order 7.*

Lean: `Conway07.no_aut_seven` ✓

uses: thm:reduction, thm:encoding-complete, thm:coverage, fact:certificates

The proof is a chain of three reductions. First, group theory reduces the statement to the non-existence of a graph invariant under one *fixed* permutation σ_0 of order 7 (§2). Second, such invariant graphs are encoded as Boolean valuations satisfying an explicit propositional formula Φ ; the encoding is proved *complete*: if an invariant graph exists, Φ is satisfiable (§3, §4). Third, Φ is proved unsatisfiable by a partition of the valuation space into 98,536 cells, each refuted by a checked certificate (§5, §6).

Chapter 2

From an order-seven automorphism to a canonical one

Lemma 3 (Cauchy step). *If $7 \mid |\text{Aut } \Gamma|$ then $\text{Aut } \Gamma$ contains an element of order 7.*

Lean: `exists_prime_orderOf_dvd_card` ✓

Lemma 4 (Fixed points of an order-seven automorphism). *Let Γ be an $\text{srg}(99, 14, 1, 2)$ and let π be an automorphism of order 7. Then π has exactly one fixed vertex, and its cycle type is $1 + 7 \cdot 14$: one fixed point and fourteen 7-cycles.*

Lean: `Conway07.card_filter_fixed_eq_one`, `Conway07.cycleType_eq_replicate` ✓

uses: `def:srg`, `lem:cauchy`

Proof sketch. Since $\text{ord } \pi = 7$ is prime, every cycle of π has length 1 or 7, so $|\text{Fix } \pi| \equiv 99 \equiv 1 \pmod{7}$. A counting argument on common neighbours excludes $|\text{Fix } \pi| \geq 2$: the fixed set induces a subgraph in which the strong regularity parameters $(\lambda, \mu) = (1, 2)$ force contradictory neighbour counts modulo 7. Hence exactly one fixed vertex, and $99 = 1 + 7 \cdot 14$ determines the cycle type. $\square$

Definition 5 (The canonical permutation). Let σ_0 be the permutation of $\{0, \dots, 98\}$ fixing 0 and acting on each block $\{1 + 7i, \dots, 7 + 7i\}$ ($0 \leq i \leq 13$) as the cyclic rotation $1 + 7i + j \mapsto 1 + 7i + (j + 1 \bmod 7)$. Its cycle type is $1 + 7 \cdot 14$.

Lean: `Conway07.sigma0`, `Conway07.sigma0_cycleType` ✓

Theorem 6 (Reduction to σ_0). *If some $\text{srg}(99, 14, 1, 2)$ has an automorphism of order 7, then some $\text{srg}(99, 14, 1, 2)$ on the vertex set $\{0, \dots, 98\}$ is invariant under σ_0 . Consequently, if no σ_0 -invariant $\text{srg}(99, 14, 1, 2)$ exists, then $7 \nmid |\text{Aut } \Gamma|$ for every $\text{srg}(99, 14, 1, 2)$ Γ .*

Lean: `Conway07.exists_sigma0_invariant`, `Conway07.seven_not_dvd_card_aut_of_no_sigma0_invariant` ✓

uses: `lem:fixed`, `def:sigma0`

Proof sketch. Transport Γ to $\{0, \dots, 98\}$ along any bijection. By Lemma 4 the resulting automorphism π has the same cycle type as σ_0 ; two permutations of equal cycle type are conjugate in the symmetric group, say $\tau\pi\tau^{-1} = \sigma_0$. The relabelled graph $\tau(\Gamma)$ is again an $\text{srg}(99, 14, 1, 2)$ and is σ_0 -invariant. $\square$

Chapter 3

The orbit quotient and the propositional encoding

Lemma 7 (Orbit structure of pairs). *The cyclic group $\langle \sigma_0 \rangle \cong \mathbb{Z}/7$ acts on the $\binom{99}{2} = 4851$ unordered pairs of vertices with exactly 693 orbits, each of size 7.*

Lean: `Conway07.orbitOf`, `Conway07.numOrbits_eq` ✓

uses: `def:sigma0`

Proposition 8 (Invariant graphs are orbit functions). *A graph G on $\{0, \dots, 98\}$ is σ_0 -invariant if and only if its edge indicator descends to the orbit space: there is $\omega: \{0, \dots, 692\} \rightarrow \{0, 1\}$ with $u \sim_G v \iff \omega(\text{orb}(u, v)) = 1$ for all $u \neq v$.*

Lean: `Conway07.invariant_iff_exists_orbitFun` ✓

uses: `lem:orbits`

Definition 9 (The formula Φ). Φ is a propositional formula in conjunctive normal form over variables $x_1, \dots, x_{693}$ (one per pair-orbit) together with auxiliary counter variables, whose clauses express, for the graph G_ω associated to a valuation ω of $x_1, \dots, x_{693}$:

1. *regularity*: every vertex has exactly 14 neighbours;
2. *common-neighbour counts*: for each orbit representative pair (u, v) , the number of common neighbours of u, v equals 2 minus the adjacency bit of (u, v) (i.e. $\lambda = 1, \mu = 2$);
3. *normalisation*: the neighbourhood of the fixed vertex 0 is the union of the first two 7-blocks;
4. *minimality*: the word $\omega \in \{0, 1\}^{693}$ is lexicographically minimal in its orbit under the 101 relabelling generators of Definition 12.

The cardinality conditions are rendered as clauses through two standard counting networks (a sequential counter and a modulo-9 totalizer); the minimality conditions through chained comparisons. Φ has 432,882 variables and 1,131,708 clauses.

Lean: `Conway07.Encoder.mk07Cnf` ✓

Computed Fact 10 (Byte identity). The formula generated inside Lean equals, byte for byte, the distributed file `o7.cnf` against which all certificates are checked.

Lean: `Conway07.mk07Cnf_eq` ✓

uses: `def:phi`

Proposition 11 (Soundness of the counting networks). *For every valuation of the orbit variables satisfying the arithmetic conditions (1)–(2) of Definition 9, the auxiliary counter variables can be extended so that all counting clauses are satisfied.*

Lean: `Conway07.Encoder.preLex_sat, Conway07.Encoder.lex_section_trip` ✓

uses: `def:phi`

Proof sketch. Each counting network is verified by a Hoare-style invariant on its constructed clauses: the sequential counter maintains the running count of true inputs; the totalizer maintains, for each internal node, a unary representation of the number of true leaves below it, split modulo 9 into quotient and remainder registers. Witness valuations for the auxiliary variables are constructed by structural induction over the emission of the clauses. $\square$

Chapter 4

Completeness of the normalisation

Definition 12 (Relabelling generators). Let $N \leq S_{99}$ be generated by: the block transposition exchanging 7-blocks 0, 1; the transpositions of adjacent blocks $i, i+1$ for $2 \leq i \leq 12$; the rotations of each block; and the five multiplier permutations $j \mapsto aj \bmod 7$ ($2 \leq a \leq 6$) applied simultaneously to all blocks. Each $\rho \in N$ normalises $\langle \sigma_0 \rangle$, hence acts on the orbit space and thus on words $\{0, 1\}^{693}$ by $(\rho \cdot \omega)(o) = \omega(\rho^{-1}o)$. The encoding uses 101 fixed generators of this action.

Lean: `Conway07.Encoder.lexPerms`, `Conway07.Encoder.orbitPerm` ✓

uses: `lem:orbits`

Lemma 13 (Normalisation of the fixed vertex’s neighbourhood). *Every σ_0 -invariant $\text{srg}(99, 14, 1, 2)$ is isomorphic, via a block relabelling commuting with σ_0 , to one in which the neighbourhood of the fixed vertex 0 is exactly the union of blocks 0 and 1.*

Lean: `Conway07.Encoder.sym1_seed` ✓

uses: `prop:orbitfun`, `def:relabel`

Proof sketch. The neighbourhood of 0 is σ_0 -invariant and of size 14, hence a union of exactly two 7-blocks; a permutation of block indices carries these two blocks to positions 0, 1, and any such permutation commutes with σ_0 and preserves strong regularity (transport along a graph isomorphism). $\square$

Theorem 14 (Completeness of minimality). *Every σ_0 -invariant $\text{srg}(99, 14, 1, 2)$ admits an isomorphic copy whose orbit word ω satisfies conditions (3)–(4) of Definition 9: the normalisation of Lemma 13 and $\omega \preceq \rho \cdot \omega$ (lexicographic order) for each of the 101 generators ρ .*

Lean: `Conway07.Encoder.lexLeaderComplete` ✓

uses: `lem:sym1`, `def:relabel`

Proof sketch. Interpret words in $\{0, 1\}^{693}$ as binary expansions of natural numbers. Among all words reachable from the normalised ω by the generator action (a set closed under each generator and preserving both strong regularity and the normalisation), choose one of minimal numerical value; the well-ordering of $\mathbb{N}$ provides it. For each generator ρ the word $\rho \cdot \omega_{\min}$ is again reachable, so $\text{val}(\omega_{\min}) \leq \text{val}(\rho \cdot \omega_{\min})$, and numerical order refines lexicographic order on fixed-length words. $\square$

Theorem 15 (Completeness of the encoding). *If a σ_0 -invariant $\text{srg}(99, 14, 1, 2)$ exists, then Φ is satisfiable.*

Lean: `Conway07.Encoder.encodes_o7` ✓

uses: `prop:orbitfun`, `prop:counters`, `thm:lex-complete`, `fact:bytes`

Chapter 5

The partition of the valuation space

Definition 16 (Sign prefixes). Fix the sequence of variables $x_{15}, x_{16}, \dots, x_{214}$. A *cell* is specified by a word $s \in \{0, 1\}^{\leq 200}$: the set of valuations assigning $x_{14+j+1} = s_j$ for $1 \leq j \leq |s|$. A family S of words is *prefix-free* if no member is a proper initial segment of another.

Lean: `Conway07.cubeOfSigns, Conway07.PrefixFree` ✓

Lemma 17 (Kraft equality and maximality). A *prefix-free family* $S \subseteq \{0, 1\}^{\leq D}$ with $\sum_{s \in S} 2^{D-|s|} = 2^D$ is a *maximal antichain of the binary tree*: every $w \in \{0, 1\}^D$ has a (necessarily unique) member of S as a prefix. Hence the corresponding cells partition the valuation space.

Lean: `Conway07.covers_of_prefixFree_kraft` ✓

uses: `def:prefix`

Theorem 18 (Refutation by cells). Let S be prefix-free with full Kraft mass as above. If for every $s \in S$ the formula $\Phi \wedge (\text{cell } s)$ is unsatisfiable, then Φ is unsatisfiable.

Lean: `Conway07.unsat_of_prefix_cover, Conway07.unsat_of_checked_cover` ✓

uses: `lem:kraft`

Computed Fact 19 (The distributed family). The distributed family of 98,536 words (of lengths between 12 and 200) is prefix-free and has full Kraft mass 2^{200} ; this is established by executing the checker of Theorem 18 on the embedded data.

Lean: `Conway07.o7Tiling_checkCover` ✓

uses: `def:prefix`

Chapter 6

Certificates

Definition 20 (Clausal derivations). A *clausal derivation* from a CNF F is a sequence of clauses, each of which follows from F and the preceding clauses by reverse unit propagation (a restricted form of resolution), ending in the empty clause; such a derivation witnesses unsatisfiability of F . The LRAT format additionally annotates each step with the indices needed to check it in linear time.

Computed Fact 21 (The external hypothesis). For each of the 98,536 cells there is a clausal derivation refuting $\Phi \wedge (\text{cell } s)$. Each derivation was checked by `cake_lpr`, an LRAT checker formally verified (in HOL4) down to the machine code that executes; verification was performed on the distributed files, whose hashes are pinned. One derivation (cell index 4092) is additionally replayed inside Lean’s kernel by the verified checker of the Lean standard library, eliminating even the translation gap for that sample.

Lean: `Conway07.cube4093_unsat`

uses: `def:lrat`, `fact:bytes`, `fact:tiling`

Assembly of Theorem 2. By Fact 21 and Theorem 18, Φ is unsatisfiable. By Theorem 15, no σ_0 -invariant $\text{srg}(99, 14, 1, 2)$ exists. By Theorem 6, no $\text{srg}(99, 14, 1, 2)$ has automorphism group of order divisible by 7. $\square$

Trusted base

The formal proof rests on: the Lean 4 kernel and mathlib; nineteen named computations executed by Lean’s compiled evaluator (each a finite check over the embedded data, including Facts 10 and 19); the HOL4-verified checker for Fact 21; and the identity of the two files (`o7.cnf` and the cell list) shared between the Lean development and the certificate checking — by construction when both halves are run from one copy of the repository.